A Appendix

S1.1 Detailed Knowledge Graph Generation

In this section, we explain each step in our graph generation process, illustrating how unstructured chemical text is transformed into a structured representation suitable for downstream tasks.

S1.1.1 Text Preprocessing

Our preprocessing pipeline is designed to isolate the most informative portions of ChemRxiv articles and prepare them for entity and relation extraction. First, we retrieved all ChemRxiv articles whose licenses permit redistribution via the ChemRxiv API. Using regular expressions, we extracted the introduction section of each paper, as this typically focuses on concise, objective statements about chemical phenomena. From each introduction, we extracted the first 500 words—this limit balances coverage of key background information against the risk of including less relevant or overly general text. To avoid fragmenting paragraphs, we ensured that no paragraph was split across chunks. Finally, we segmented each introduction into contiguous chunks of up to 128 words, preserving natural sentence boundaries. This segmentation reduces the input length for subsequent processing steps, ensuring that each chunk remains within the token limits of our downstream models while retaining semantic coherence.

S1.1.2 Node Extraction

Once the text is segmented, we identify chemical entities and their interrelations. We apply a named entity recognition (NER) model based on the PubMedBERT architecture, fine-tuned on multiple chemical datasets, to detect candidate entities within each 128-word chunk. To further improve precision and eliminate spurious mentions, we refine the NER output using OpenAI's `gpt-4o`, prompting it with a few shots to verify whether each detected span indeed corresponds to a chemically meaningful entity and to standardize entity labels (for instance, converting "MeOH" to "methanol"). Below, we showed the prompt for Entity verification.

> You are a chemistry expert specializing in entity recognition. Your task is to **validate**

and filter the extracted entities, ensuring they are **chemically meaningful** based on the provided text. Remove any irrelevant terms, including general descriptors, numerical values, reaction conditions, and vague terms.

Entities Extracted by NER:
`{entities}`

Text for Context:
`{text}`

Criteria for Valid Entities:

3 Chemical compounds (e.g., "HCl", "Sodium hydroxide", "Ethanol", "Benzene")
3 Chemical elements (e.g., "Carbon", "Oxygen", "Cesium")
3 Specific catalysts, solvents, reagents (e.g., "Cs_2CO_3", "Toluene", "Palladium")

Remove the Following Types of Entities:

× Generic terms (e.g., "Reaction", "Solvent", "Acid", "Base", "Solution")
× Experimental conditions (e.g., "pH", "Temperature", "2 M", "Strong acid")
× Measurement terms (e.g., "X-ray diffraction", "NMR")
× General descriptors (e.g., "High concentration", "Low efficiency")

Output Format:
Return only a **Python list** of valid chemical entities, with no explanations, markdown, or extra formatting.

S1.1.3 Edge Extraction

After obtaining a vetted set of entities, we extract pairwise relations using the same `gpt-4o` instance. For each pair of entities co-occurring within a chunk, we prompt the model with a few shots to classify or generate the nature of their relationship, producing triplets of the form (entity_A, relation, entity_B). The result of this step is a set of entity nodes (chemical names) and directed edges (relation labels) extracted directly from the text, forming a raw triplet collection that reflects the functional associations present in the chemical literature. Below we showed the prompt for Edge Extraction.

You are an expert in chemical text analysis. Your task is to extract **only chemically**

meaningful relationships between a given set of entities from the provided text.

Guidelines for Relation Extraction:

1. **Entity Matching:** Consider only the entities provided in the given set. If an entity appears in the text but has no meaningful chemical relationship with another entity in the set, ignore it.
2. **Chemically Significant Relations Only:** Extract relations that describe actual **chemical interactions, transformations, or properties** (e.g., "reacts with," "catalyzes," "dissolves in," "produces").
3. **Factual Relations:** Only extract factual relations. Avoid observations, opinions, and findings.
4. **Tuple Format:** Output extracted facts in the form of **(entity1, relation, entity2)**.
5. **Avoid Generic Relations:** Exclude weak relations like "is," "are," "exists," "relates to." Focus on **specific interactions**.

Valid Relation Types (Examples):

3 "reacts with"
3 "catalyzes"
3 "binds to"
3 "dissolves in"
3 "oxidizes"
3 "inhibits"
3 "precipitates with"
3 "acts as a solvent for"
3 "is synthesized from"

Avoid These Weak Relations: Exclude relations such as "is," "are," "has," "exists."

Entities Provided:
`{entities}`

Text:
`{text}`

Extract at most {`max_facts`} factual statements.

Output Format:
Provide the output as a **Python list of tuples**, containing only the extracted relationships without any code formatting, backticks, or markdown.

Example Output:

– [("HCl", "dissolves in", "Water"), ("HCl", "reacts with", "Sodium hydroxide")]

S1.1.4 Knowledge Enrichment

To enrich each node with descriptive metadata and resolve naming ambiguities, we retrieved supplemental information from two external resources: Wikipedia and PubChem. From Wikipedia, we extracted the introductory summary of each entity's page, providing a concise description of its common usage, historical context, or primary function. From PubChem, we obtained the official compound name (Record Title) alongside additional names and identifiers drawn from the "Names and Identifiers" section. We retrieved a textual description from the "Record Description" heading, summarizing key properties or applications. Safety annotations, including hazard statements or pictograms, were collected from the "Chemical Safety" subsection. Structural information was captured in the form of the canonical SMILES string under "Computed Descriptors," and the molecular formula was fetched from the "Molecular Formula" field. Finally, we extracted computed physicochemical properties—such as molecular weight, topological polar surface area, and log P values—from the "Computed Properties" list within "Chemical and Physical Properties." All of these metadata fields were stored as additional text and added as the external info for each of the nodes.

S1.1.5 Graph Generation

At this stage, we constructed the graph by forming triplets of the form (node, edge, node). Each node was linked to the original text segment from which it was extracted and, if available, to its corresponding external metadata. Likewise, each edge was associated with the specific text chunk that produced it. In this way, both nodes and edges maintain direct references to their source text, ensuring traceability throughout the knowledge graph.

S1.2 Detailed Question Generation

S1.2.1 Path Sampling from the Knowledge Graph

To generate multi-hop questions, we first sampled paths of varying lengths from the constructed knowledge graph using a randomized breadth-first search (BFS) path sampling algorithm. During path sampling, we enforced that each edge in a sampled path originates from a distinct source text, thereby encouraging the integration of information from multiple, separate context passages. Concretely, a path of length K comprises $K + 1$ entities and traces through K different source texts extracted from the original ChemRxiv database. By requiring unique sources for each edge, we ensure that correct solutions must draw on evidence scattered across several documents rather than a single paragraph.

S1.2.2 One-Hop Question Formulation

Adopting a bottom-up approach, we generated individual one-hop questions for each edge along a sampled path. For every triplet $(\text{entity}_1, \text{relation}, \text{entity}_2)$, we framed a question whose answer is entity_1 by asking, "Which entity holds the relation relation to entity_2" If the initial phrasing lacked sufficient specificity or clarity, we invoked a language model (OpenAI's `o3-mini`) to augment the prompt with metadata drawn from the original text segment, such as contextual phrases or qualifying details. This enrichment step ensures that each one-hop question remains clear, precise, and answerable from the associated source text alone. Below is the prompt for generating the one hop questions.

> You are given a text along with an entity and its relation to another entity.

Entity 1: {`entity1`}
Relation: {`relation`}
Entity 2: {`entity2`}
Text: {`text`}

Information about Entity1: {`entity1_meta` if `entity1_meta` else `None`}

Your task is to generate a factual question whose answer is Entity1.
The question should ask for the entity that has the specified relation to Entity2.
Do not mention the answer (which is Entity1) in the question.
Ensure that the question is factual and can be answered solely based on the given text and the information about Entity1.
Do not refer to sections such as "Abstract," "Table 1," "in the text," or "in the article."

If Entity1 and relation are not specific enough (i.e., multiple answers are possible), add descriptions from the text or from the information about Entity1 to make it specific so that Entity1 is the only answer.

Return a dictionary without any code formatting, backticks, or markdown, with keys `\q`" and `\a`".

S1.2.3 Multi-Hop Question Aggregation

Once all one-hop questions for a path were vetted, we combined them into a single multi-hop question by prompting OpenAI's `o3-mini` model, with a few shots. The final aggregated question is constructed by starting with the sub-question corresponding to the last edge (i.e., the entity nearest to the "tail" of the path) and then chaining backward through each preceding entity until reaching entity_1 of the first relation. In other words, if the sampled path is

$$(\text{entity}_1 \xrightarrow{\text{relation}_1} \text{entity}_2),\ (\text{entity}_2 \xrightarrow{\text{relation}_2} \text{entity}_3),\ \ldots,\ (\text{entity}_K \xrightarrow{\text{relation}_K} \text{entity}_{K+1}),$$

the aggregated question asks first about entity_{K+1} (the final tail), then uses its answer as context for the penultimate question, and so on, so that the final answer corresponds to entity_1. This reverse-chaining structure ensures that the multi-hop prompt leads directly to the original target node while preserving logical flow. Below we showed the prompt for generating the multi hop question.

You are given multiple factual questions and their answers that are logically connected.

Your task is to chain them into a single, coherent multi-hop question that requires multiple reasoning steps.
Ensure that the (only) answer is the answer to the first question, and the question naturally follows from the facts given.
You have to start from the last generated question and build up a single multi-hop question so it aggregates them all and the answer is the answer to the first question.
None of the answers to any of the questions should be in the generated question.

Here is an example:
Example:
Q1: What is oxidized to form Carbon Dioxide?
A1: Methane
Q2: What is used in Photosynthesis?
A2: Carbon Dioxide
Q3: What produces Oxygen?
A3: Photosynthesis

Multi-hop question:
Q: What is oxidized to produce a substance that is used in a process that results in Oxygen?
A: Methane

Here are the generated questions and answers:
`{formatted_qas}`

Return a python dictionary without any code formatting, backticks, or markdown, with keys `"q"` (multi-hop question) and `"a"` (final answer).

S1.2.4 Verification and Filtering

During verification, we first reviewed each one-hop question for clarity, chemical relevance, and direct answerability based on its corresponding source text. Below the prompt for one-hop question evaluation is shown.

You are a chemistry expert. Your task is to determine if the given question is a factual chemistry question, unambiguous (has only one answer), and answerable based on the provided context. A factual question must be based on actual chemical properties, reactions, or experimentally verified principles and must be strictly related to chemistry. An answerable question should be solvable based on the given context and must not be open-ended or have multiple correct answers. MAKE SURE THE QUESTION HAS ONLY ONE CORRECT ANSWER. There shouldn't be any other entity except for the given answer that could be another answer.

Question:
`{question}`

Answer:
`{answer}`

Context:
`{context}`

Please analyze the context and verify if the question is factual, unambiguous, and answerable. If the question is factual, has only one correct answer, is strictly related to chemistry, and can be answered based on the context, return "yes." Otherwise, return "no."

Examples of Factual Chemistry Questions:
"What dissolves in water and evaporates at 0 °C?"
"What catalyst is used in the reaction between A and B?"

Examples of Non-Factual or Ambiguous Chemistry Questions:
"What is the song of Nirvana that is a chemical entity?"
"What chemical entity and structural unit form the layered hydroxide structures with intercalated water ions used in battery materials and OER catalysis?" ($M(OH)_6$ and -$Ni(OH)_2$ are valid answers)
Questions that have multiple possible correct answers or are not strictly related to chemistry.

Next, the multi-hop question underwent an additional evaluation step to confirm that the logical chain—from the final sub-question back to the first—correctly guides a reader (or model) to $entity_1$. We employed an LLM-based verification process to assess factual accuracy, answerability given available context and metadata, and overall coherence among sub-questions. Feedback from domain experts was continuously incorporated into the prompt templates to refine verification accuracy. Any questions—either one-hop or multi-hop—that were answered incorrectly by all evaluated models were excluded from the benchmark to minimize ambiguity and ensure high-quality, unambiguous reasoning tasks. Below the prompt for evaluating the whole path is shown

You are a chemistry expert. Your task is to determine if the given question is a factual chemistry question and answerable based on the provided path.

Path Information:
`{path_text}`

Question:
`{question}`

Answer:
`{answer}`

Please analyze the path and verify if the question is a factual chemistry question and can be answered based on the given path. A factual question must be based on actual chemical properties, reactions, or experimentally verified principles. An answerable question should be solvable based on the given path. If the question is factual and answerable, return "yes". If it contains speculation, opinions, or lacks verifiable chemical grounding, or it is not solvable, return "no".

Examples of Factual Chemistry Questions:
"What dissolves in water?"
"What catalyst is used in the reaction between A and B?"
"Which compound undergoes oxidation in this reaction?"
"What product is formed when sodium reacts with chlorine?"

Examples of Non-Factual Chemistry Questions:
"Why do some scientists think this reaction is inefficient?"
"What is the best solvent for this reaction?"
"Is this reaction useful in industry?"
"Do you think this compound is a good catalyst?"

Provide only "yes" or "no" as your response.

S1.2.5 Short-Answer Design and Rationale

To minimize the impact of writing style and summarization on accuracy evaluation, all questions were deliberately designed to elicit short, concise answers. Answering a multi-hop question requires decomposing it into its constitutive one-hop sub-questions, retrieving each intermediate answer, and then combining them to arrive at the final answer. Even when full context is available, a correct response cannot be produced if a model fails to infer and integrate multiple pieces of knowledge. By keeping answers brief and focusing on discrete factual steps, we ensure that performance evaluation reflects a model's ability to conduct multi-hop inference rather than its capacity to paraphrase or generate lengthy explanations.

S1.3 Rejected Questions

There were a couple of reasons that we identified for rejecting of some questions from the benchmark. One was having multiple valid answers, and second was the answer was present in the question. Some examples

Q1:

Context:
Researchers have developed an anode material based on $NiCo_rGO$ (Nickel–Cobalt–reduced Graphene Oxide). In some variants, the $NiCo_rGO$ is further decorated with palladium (Pd) nanoparticles to enhance catalytic performance.

Question:
Which component in the electrode structure functions as a catalyst at the anode when incorporating decorated $NiCo_rGO$?

Issue:
The question is declined due to ambiguity — two distinct answers are technically correct based on the variant of the material:

- If the material is **$NiCo_rGO$** without Pd, the catalyst is **Nickel (Ni)**.
- If the material is **Pd-decorated $NiCo_rGO$**, the catalyst is **Palladium (Pd)**.

Since the phrasing of the question does not clearly disambiguate which material variant is being used, it leads to multiple valid interpretations. Therefore, it cannot be accepted as a single-answer question.

Q2:

Context:
Hypervalent iodine compounds such as diaryliodonium salts are widely used as electrophilic arylation reagents. According to the source text, these salts are employed in both **transition metal-catalyzed** and **metal-free** arylation reactions. These reactions can be used to functionalize aromatic compounds, including halogen-substituted analogues like CIMPPC, by replacing hydrogen atoms.

Question:
Which type of arylation reaction that employs electrophilic arylation reagents utilizes diaryliodonium salts in hypervalent iodine chemistry?

Expected Answer:
Transition metal-catalyzed arylations

Reason for Rejection:
The question is declined due to the presence of **multiple valid answers**. The source explicitly states that diaryliodonium salts are used in:

- **Transition metal-catalyzed arylations**, and
- **Metal-free arylations**.

Both are equally valid interpretations of the question. Without further constraints or clarification, the question has more than one correct answer and does not meet the single-answer requirement.

S1.4 Detailed Performance Based on Context Availability

Figure S1 shows that model performance is strongly influenced by whether context is provided in the input. In particular, Claude 3.7 with extended thinking achieved a correctness rate of 84% with context, whereas `o3-mini` recorded the highest correctness rate (48%) when the context was absent. Note that `o3-mini` was primarily used to generate the questions, which may have introduced a slight bias resulting in its minor improvement in correctness. Figures S2 and S3 illustrate the token usage and latency of the models, respectively.